Custom Q-Learning Agent for Taxi-v3

1. Problem description

The task is to navigate a taxi through a 5 x 5 grid, collect a passenger, and deliver them to a designated destination. Internal walls constrain movement. Taxi-v3 is created with gymnasium.make(), using Gymnasium 1.2.1 and its default deterministic dynamics.
State encoding: ((row * 5 + column) * 5 + passenger) * 4 + destination. There are 500 encoded states: 25 taxi positions, five passenger positions (four landmarks or inside the taxi), and four destinations. Only a subset is reachable. The reset distribution has 300 valid starting states.
Six actions: south, north, east, west, pickup, and dropoff. Rewards are -1 for an ordinary step, -10 for an illegal pickup/dropoff, and +20 for successful delivery. Delivery terminates the episode; the wrapper truncates it at 200 steps.

2. Objective and scope

Maximize expected discounted return while learning a short, reliable delivery policy. No pretrained policy, RL framework, action masking, planning model, or hand-coded route is used. NumPy stores the learned action values; Gymnasium supplies only the environment. The assignment requires source code, an executed notebook, saved model, rendered demonstration, convergence analysis, and hyperparameter sensitivity. All artifacts are generated by reproducible scripts in the repository.

Methodology and implementation

3. Q-learning update

Q(s,a) <- Q(s,a) + alpha * [r + gamma * (1 - terminal) * max_b Q(s_next,b) - Q(s,a)].
This off-policy temporal-difference rule moves the current estimate toward the observed reward plus the best estimated future value. The Q-table starts at zero. Terminal delivery has target r only. Time-limit truncation ends the rollout but retains the bootstrap because it is an external limit, not a terminal MDP state.

4. Exploration and architecture

At episode e (zero-based), epsilon = max(0.01, 1.0 * 0.999^e). With probability epsilon the agent samples uniformly from all six actions; otherwise it selects a maximal Q-value, breaking ties randomly during training. Evaluation uses deterministic argmax and epsilon zero.
agent.py owns the table, action selection, Bellman update, and model persistence. utils.py implements training, independent evaluation, plotting, and RGB rendering. main.py orchestrates experiments and writes raw CSV logs and JSON metrics. report.py and build_notebook.py generate the remaining deliverables.

5. Reproducibility and model selection

Training seeds are 0, 1, and 2. Each learning-rate setting uses identical episode budgets and schedules. At 1,000-episode intervals, greedy policies are assessed on reset seeds 20000-20099. The learning rate is selected by final validation return averaged across training seeds; a tie selects the lower rate. Deployment always uses training seed 0.
After selection, every policy and a uniform-random baseline are evaluated on 500 separate reset seeds 30000-30499. The GIF uses another reset seed (40000). Validation rollouts do not update the Q-table or consume the training RNG. The test set does not select hyperparameters.

Experimental setup and measured results

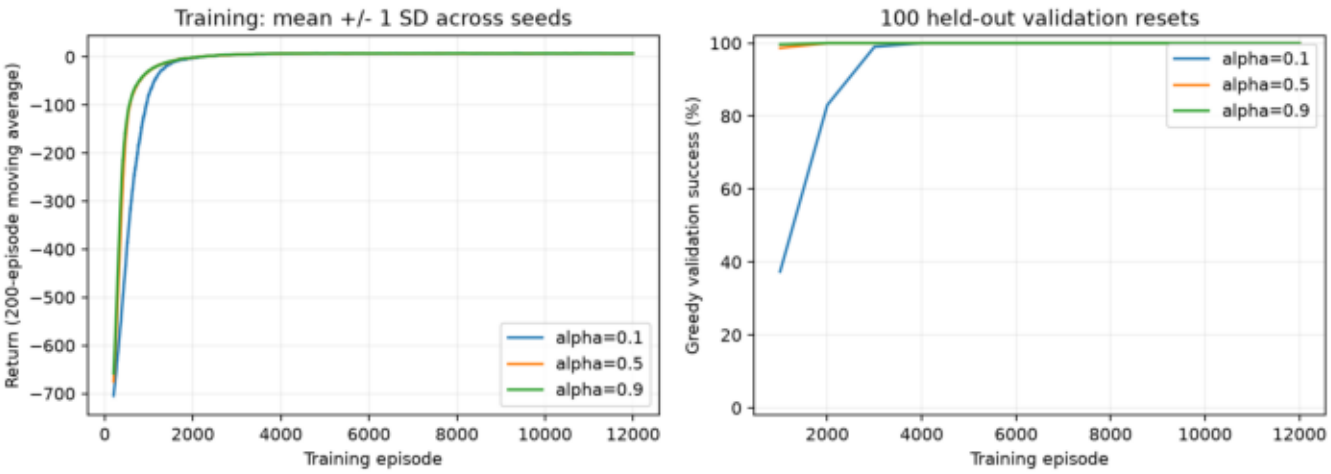
6. Final hyperparameters

All settings except alpha are held fixed. The sensitivity sweep is alpha in {0.1, 0.5, 0.9}.

Hyperparameter	Final value
alpha	0.1
gamma	0.99
epsilon_start	1.0
epsilon_min	0.01
epsilon_decay	0.999
episodes	12000
Q-table	500 x 6 float64
Maximum steps	200

7. Held-out performance

alpha=0.1: return 8.030 +/- 0.000; success 100.00%; steps 12.97
alpha=0.5: return 7.990 +/- 0.018; success 100.00%; steps 13.01
alpha=0.9: return 8.019 +/- 0.010; success 100.00%; steps 12.98
Values are mean +/- population SD of three seed-level mean returns; these are descriptive spreads, not confidence intervals. The chart shows smoothed training returns and independent greedy validation success.



Discussion, limitations and conclusion

8. Baseline and selected agent

The selected alpha=0.1, seed-0 model obtained mean return 8.030, success 100.0%, mean length 12.97, and 0.000 illegal actions per episode on 500 test resets. The uniform-random baseline obtained return -773.024, success 4.2%, and mean length 197.62.
The saved GIF records successful delivery in 10 steps with return 11. It illustrates one rollout; the larger test set measures reliability.

9. Convergence and sensitivity

alpha=0.1: first observed 100% validation success at episodes 4000, 3000, 4000 (seeds 0/1/2).
alpha=0.5: first observed 100% validation success at episodes 2000, 2000, 1000 (seeds 0/1/2).
alpha=0.9: first observed 100% validation success at episodes 2000, 1000, 1000 (seeds 0/1/2).
These checkpoint milestones measure observed learning speed, not a proof of convergence. A nonzero exploration floor leaves occasional penalties in training even after the greedy policy becomes reliable. The final test score alone can conceal differences in early sample efficiency.

10. Failure modes and limitations

Early exploration incurs illegal pickup/dropoff penalties and wall collisions. Unlearned or tied states can cause greedy loops until the time limit. Constant learning rates and a finite budget do not establish theoretical convergence. Three seeds give limited uncertainty estimates, and 500 sampled resets are not exhaustive verification. Only alpha was varied; discount and exploration sensitivity remain future work. This small deterministic benchmark does not demonstrate generalization to traffic, continuous control, or changing maps.

11. Conclusion

From-scratch tabular Q-learning learns a delivery policy on Taxi-v3. Separate validation and test rollouts, a random baseline, multiple training seeds, raw logs, and a rendered demonstration make the empirical findings auditable.

References

Farama Foundation: gymnasium.farama.org/environments/toy_text/taxi/
Farama Foundation: gymnasium.farama.org/tutorials/gymnasium_basics/handling_time_limits/
Sutton & Barto (2018), Reinforcement Learning: An Introduction, 2nd ed., Chapter 6.
Environment specification verified against the installed Gymnasium 1.2.1 source; current online documentation may describe newer versions.